

1. Concurrency Control Motivation

ACID Isolation

- Coordinates concurrent execution to avoid 4 major anomalies:
- **Dirty Read (WR):** Reading uncommitted data that later aborts.
  - **Lost Update (WW):** Two writes overwrite without intermediate visibility.
  - **Unrepeatable Read (RW):** Reading different values in same txn.
  - **Phantom Read:** Range query tuple count changes via inserts.

2. Lock Types & Compatibility

Lock Manager

- **Binary Lock (0/1):** 0 = Unlocked, 1 = Locked. Only 1 txn allowed (Bottleneck).
- **Shared Lock S(X):** Read permission. Multiple readers allowed.
- **Exclusive Lock X(X):** Read and Write permission. Strictly 1 txn.

| Held \ Requested | Shared (S)        | Exclusive (X)    |
|------------------|-------------------|------------------|
| Shared (S)       | TRUE (Compatible) | FALSE (Conflict) |
| Exclusive (X)    | FALSE (Conflict)  | FALSE (Conflict) |

3. Two-Phase Locking (2PL) Protocol & Lock Point

Serializability Theorem

Phase 1: Growing Phase (Locks acquired only; no release)  
Lock Point: Instant when the LAST lock is acquired  
Phase 2: Shrinking Phase (Locks released only; no acquire)  
**Golden Rule:** Once a transaction releases any lock, it can NEVER acquire any new lock!

**Lock Conversion Rules:**

- **Upgrading (S → X):** Allowed ONLY in Growing Phase if T is sole reader.
- **Downgrading (X → S):** Allowed in Shrinking Phase to increase read concurrency.

**Serial Order:** Equivalent serial order equals the order of transactions reaching their **Lock Points**.

4. Master 2PL Protocol Variants Comparison Matrix

AKTU 10M Question

| Variant          | Lock Holding & Release Rule                                                                         | Deadlock-Free?         | Cascadeless?                  | Industry Role                         |
|------------------|-----------------------------------------------------------------------------------------------------|------------------------|-------------------------------|---------------------------------------|
| Basic 2PL        | Growing phase then Shrinking phase. Locks released anytime in phase 2.                              | No (Deadlock possible) | No (Cascading abort possible) | Theoretical baseline                  |
| Conservative 2PL | Pre-declares and locks ALL required items before execution starts.                                  | YES (Guaranteed!)      | No                            | Static/Embedded DBs                   |
| Strict 2PL       | All <b>Exclusive (X)</b> locks held until <b>Commit / Abort</b> . Shared locks released in phase 2. | No (Deadlock possible) | YES (Strict & Cascadeless)    | Industry Standard (PostgreSQL, MySQL) |
| Rigorous 2PL     | All <b>Shared (S)</b> and <b>Exclusive (X)</b> locks held until <b>Commit / Abort</b> .             | No (Deadlock possible) | YES (Commit Order CSR)        | Financial engines                     |

5. Basic Timestamp Ordering (TO)

Deadlock-Free

TS(T) assigned at start. Item timestamps: read\_TS(X) & write\_TS(X).

**read(X):** If  $TS(T) < write\_TS(X) \rightarrow$  ABORT T.  
Else: execute and  $read\_TS(X) = \max(read\_TS(X), TS(T))$ .  
**write(X):** If  $TS(T) < read\_TS(X)$  or  $TS(T) < write\_TS(X) \rightarrow$  ABORT T.  
Else: execute and  $write\_TS(X) = TS(T)$ .

**Advantage:** 100% Deadlock-Free (No waiting cycles!).

7. Multi-Version (MVCC) & MVTO

Non-Blocking Readers

Writes append new version  $X_k$  (val), RTS, WTS angle\$.

- Mantra:** Readers never block writers; writers never block readers!
- MVTO read(X):** Reads  $X_i$  with largest  $WTS(X_i) \leq TS(T)$ . **Read is ALWAYS granted (Never aborts)!**
- MVTO write(X):** If  $RTS(X_i) > TS(T) \rightarrow$  Abort. Else creates new version  $X_{k+1}$  with  $WTS = RTS = TS(T)$ .

9. Validation-Based Protocol (OCC) & Phantom Phenomena

The 3 Phases of OCC:

- Read Phase:** Reads DB freely; writes buffered locally in memory.
- Validation Phase:** Assigned  $TS(T)$ ; validates conflict conditions.
- Write Phase:** Local copies flushed to DB disk upon validation pass.

Condition 2:  $WritePhase(T_i) < WritePhase(T_j) \rightarrow \{ and \} RS(T_j) \cap WS(T_i) = \emptyset$

6. Thomas' Write Rule & VSR

Obsolete Writes

Modifies Basic TO for write operations:

- If  $TS(T) < read\_TS(X) \rightarrow$  ABORT and ROLLBACK T.
- If  $TS(T) < write\_TS(X) \rightarrow$  **IGNORE write, CONTINUE processing!**
- Else  $\rightarrow$  Execute write,  $write\_TS(X) = TS(T)$ .

**AKTU Theorem:** Enforces **View Serializability (VSR)** using Blind Writes, rejecting fewer writes than Basic TO!

8. MV2PL & Certify Locks

3 Lock Modes

Employs Read Lock (RL), Write Lock (WL), Certify Lock (CL):

| Held \ Req | RL  | WL  | CL |
|------------|-----|-----|----|
| RL         | YES | YES | NO |
| WL         | YES | NO  | NO |
| CL         | NO  | NO  | NO |

WL upgrades to CL at commit phase. Eliminates cascading rollbacks!

The Phantom Phenomenon:

- Predicate query count altered by concurrent dynamic tuple insertions.
- Existing row locks cannot protect non-existent records.
- Solution: Index-Range Locking** on B+ Tree leaf keys covering search range.

10. Granularity Tree Hierarchy

db → f → p → r

Tree: Database (\$db\$) → Files (\$f\_1, f\_2\$) → Pages (\$p\_{11}, p\_{12}\$) → Records (\$r\_{111}\$).

- **Fine (Record):** High concurrency, high locking overhead.
- **Coarse (File):** Low concurrency, minimal locking overhead.
- **Subtree Invariant:** Locking node \$N\$ locks all its descendants!

11. Intention Locks & 5x5 Matrix (Slide 69)

IS, IX, SIX

| Held \ Req | IS  | IX  | S   | SIX | X  |
|------------|-----|-----|-----|-----|----|
| IS         | Yes | Yes | Yes | Yes | No |
| IX         | Yes | Yes | No  | No  | No |
| S          | Yes | No  | Yes | No  | No |
| SIX        | Yes | No  | No  | No  | No |
| X          | No  | No  | No  | No  | No |

12. The 6 Mandatory Rules of MGL Protocol (Slide 70)

AKTU Exam Theorem

- **Rule 1:** Adhere strictly to 5×5 lock compatibility matrix.
- **Rule 2:** Root of tree (\$db\$) must be locked first, in any mode.
- **Rule 3 (S / IS):** Lock node \$N\$ in \$S\$ or \$IS\$ only if parent is locked in \$IS\$ or \$SIX\$.
- **Rule 4 (X / IX / SIX):** Lock node \$N\$ in \$X, IX, SIX\$ only if parent locked in \$IX\$ or \$SIX\$.
- **Rule 5 (2PL):** Lock only if no node has been unlocked (Growing phase).
- **Rule 6 (Bottom-Up):** Unlock \$N\$ only if NONE of its children are currently locked!

Direction: Locking is strictly TOP-DOWN | Unlocking is strictly BOTTOM-UP

13. Concurrent Recovery & 2PC

Distributed Atomicity

- **Concurrent Recovery:** Checkpoint records active list \$L\$. \$T\_i\$ with commit → REDO (forward). \$T\_i\$ active at crash → UNDO (backward).
- **Two-Phase Commit (2PC):**
  - **Phase 1 (Prepare):** Coordinator sends PREPARE → Nodes vote COMMIT or ABORT.
  - **Phase 2 (Commit):** If all vote COMMIT → GLOBAL\_COMMIT; else GLOBAL\_ABORT.

14. Oracle Case Study (AKTU Syllabus)

Industry Engine

- **MVCC & SCN:** System Change Number increments at commit.
- **Undo Segments:** Reconstructs Consistent Read (CR) buffer for older queries without blocking writers.
- **ORA-01555 Snapshot Too Old:** Occurs when query's required undo blocks are overwritten. Solution: Increase UNDO\_RETENTION.
- **Zero Lock Escalation:** Row locks in block ITL; never escalates.

# Module 01: Concurrency Control Foundations and Locking Basics

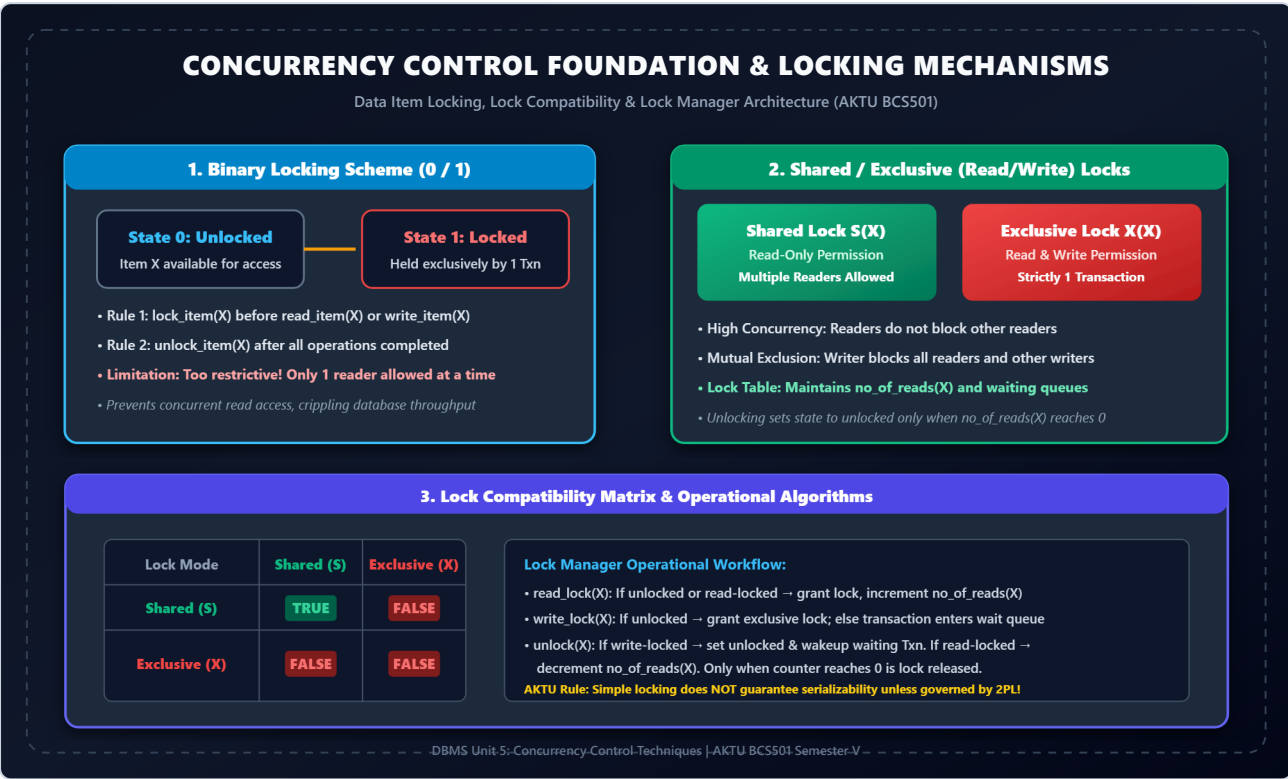


Figure 1.1: Binary vs Shared/Exclusive Locks & Lock Compatibility Architecture

## 1. Concurrency Control Overview & Motivation

Concurrency Control Manager ensures Isolation and Consistency across concurrent transactions. Without proper synchronization, interleaved operations cause Dirty Reads, Lost Updates, and Unrepeatable Reads.

## 2. Binary vs Shared/Exclusive (Read/Write) Locks

Binary locks allow only 1 transaction per item (0 = Unlocked, 1 = Locked), severely bottlenecking throughput. Two-Mode Locks support Shared mode S(X) for concurrent readers and Exclusive mode X(X) for writers.

**Compatibility Rule**  
Shared locks are mutually compatible with other Shared locks. Exclusive locks are mutually exclusive with ALL locks.

## 3. Lock Compatibility Matrix

| Mode Held \ Requested | Shared (S)        | Exclusive (X)    |
|-----------------------|-------------------|------------------|
| Shared (S)            | TRUE (Compatible) | FALSE (Conflict) |
| Exclusive (X)         | FALSE (Conflict)  | FALSE (Conflict) |

## 4. Lock Manager Internal Operations

Lock Manager maintains a hash table of locks with transaction wait queues and reader count `no_of_reads(X)` . Lock is fully released to unlocked state only when reader counter reaches 0.

# Module 02: Two-Phase Locking (2PL) Protocol and Lock Conversion

AKTU BCS501 | Unit 5: Concurrency Control Techniques

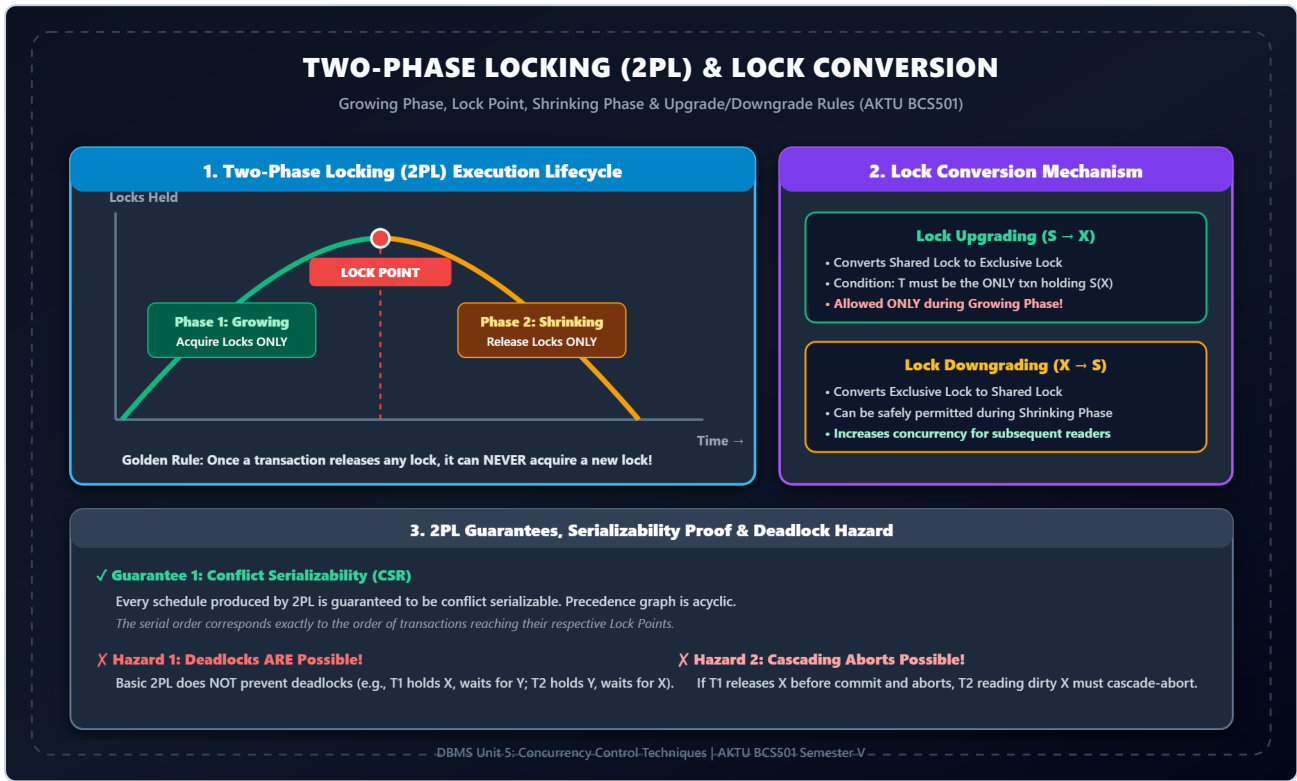


Figure 2.1: 2PL Growing Phase, Lock Point, Shrinking Phase & Conversion Rules

## 1. Two-Phase Locking (2PL) Definition

A transaction follows 2PL if all lock acquisitions precede the first unlock operation. It consists of a **Growing Phase** (acquire only) and a **Shrinking Phase** (release only).

### Lock Point Theorem

The exact timestamp when a transaction acquires its final lock is its **Lock Point**. The equivalent serial schedule order corresponds strictly to the chronological ordering of transactions' lock points.

## 2. Lock Conversion (Upgrading & Downgrading)

**Upgrading ( $S \rightarrow X$ ):** Allowed strictly during the Growing Phase if the transaction is the sole reader holding S(X).

**Downgrading ( $X \rightarrow S$ ):** Can be safely performed in the Shrinking Phase to increase read concurrency for others.

## 3. 2PL Guarantees & Potential Hazards

2PL guarantees Conflict Serializability, but does NOT prevent **Deadlocks** (cyclic lock waits) or **Cascading Aborts** (if dirty locks are unlocked early).

# Module 03: Two-Phase Locking Variants (Strict, Rigorous, Conservative)

AKTU BCS501 | Unit 5: Concurrency Control Techniques

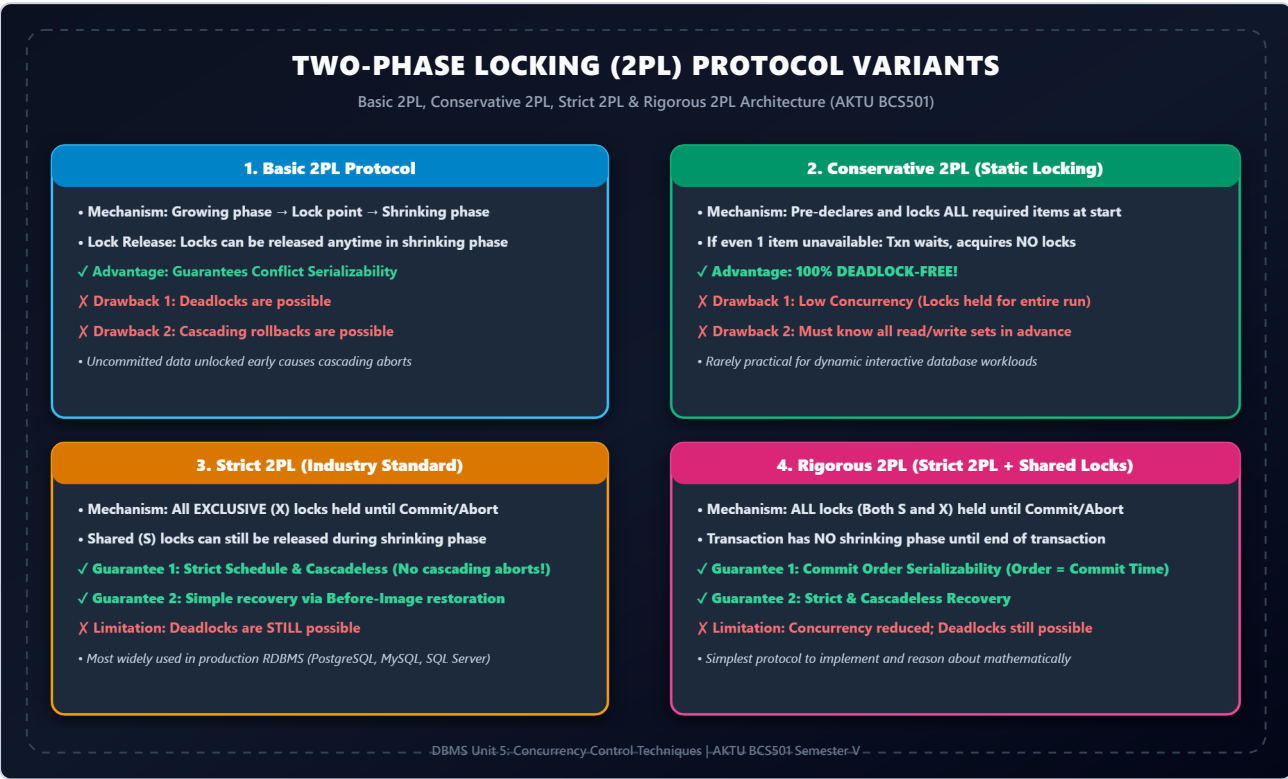


Figure 3.1: Basic, Conservative, Strict and Rigorous 2PL Protocol Architecture

## 1. Why 2PL Variants are Necessary

Basic 2PL guarantees Conflict Serializability, but allows Deadlocks and Cascading Aborts. Variants solve these operational bottlenecks.

## 2. Comparative Breakdown of Variants

| Variant          | Lock Release Timing                      | Deadlock-Free? | Cascadeless? |
|------------------|------------------------------------------|----------------|--------------|
| Basic 2PL        | Shrinking phase                          | No             | No           |
| Conservative 2PL | Pre-locks all items at start             | YES            | No           |
| Strict 2PL       | Exclusive locks held till Commit/Abort   | No             | YES (Strict) |
| Rigorous 2PL     | All locks (S & X) held till Commit/Abort | No             | YES (Strict) |

### Industry Standard: Strict 2PL

Modern RDBMS (PostgreSQL, MySQL InnoDB, Oracle) employ Strict 2PL because it guarantees strict recoverability and simple before-image rollback while maintaining high concurrency.

# Module 04: Timestamp Ordering Protocols

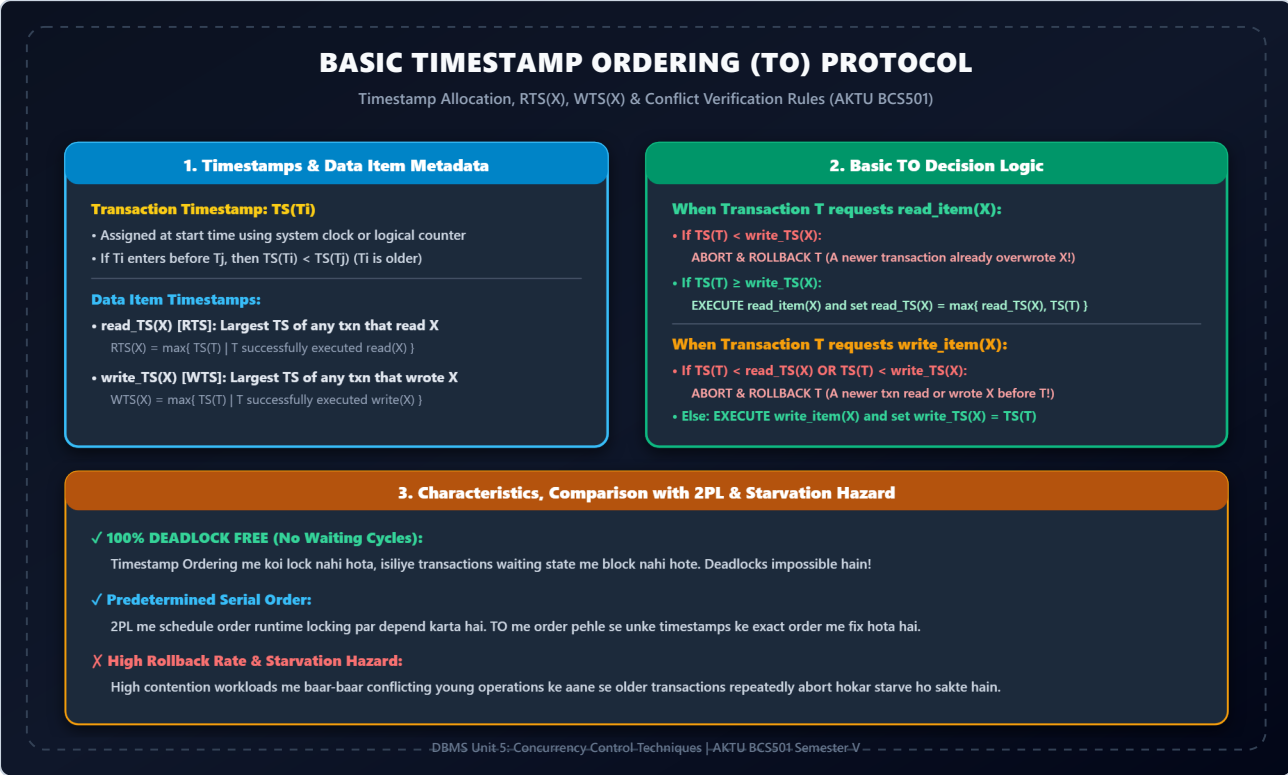


Figure 4.1: Basic Timestamp Ordering Rules & Metadata Architecture

## 1. Timestamp Assignment & Data Item Metadata

Every transaction receives a unique timestamp TS(Ti) at start. Data items maintain `read_TS(X)` (largest timestamp of any reader) and `write_TS(X)` (largest timestamp of any writer).

## 2. Basic TO Rules Summary

### Read and Write Check Conditions

- read(X):** If  $TS(T) < write\_TS(X) \rightarrow$  ABORT T. Else execute and set  $read\_TS(X) = \max(read\_TS(X), TS(T))$ .
- write(X):** If  $TS(T) < read\_TS(X)$  or  $TS(T) < write\_TS(X) \rightarrow$  ABORT T. Else execute and set  $write\_TS(X) = TS(T)$ .

## 3. Deadlock-Free Guarantee vs Cascading Aborts

Because conflicting transactions are immediately aborted rather than forced to wait in queues, Timestamp Ordering is 100% deadlock-free. However, restart overhead can be high under contention.

# Module 05: Thomas' Write Rule and View Serializability

AKTU BCS501 | Unit 5: Concurrency Control Techniques

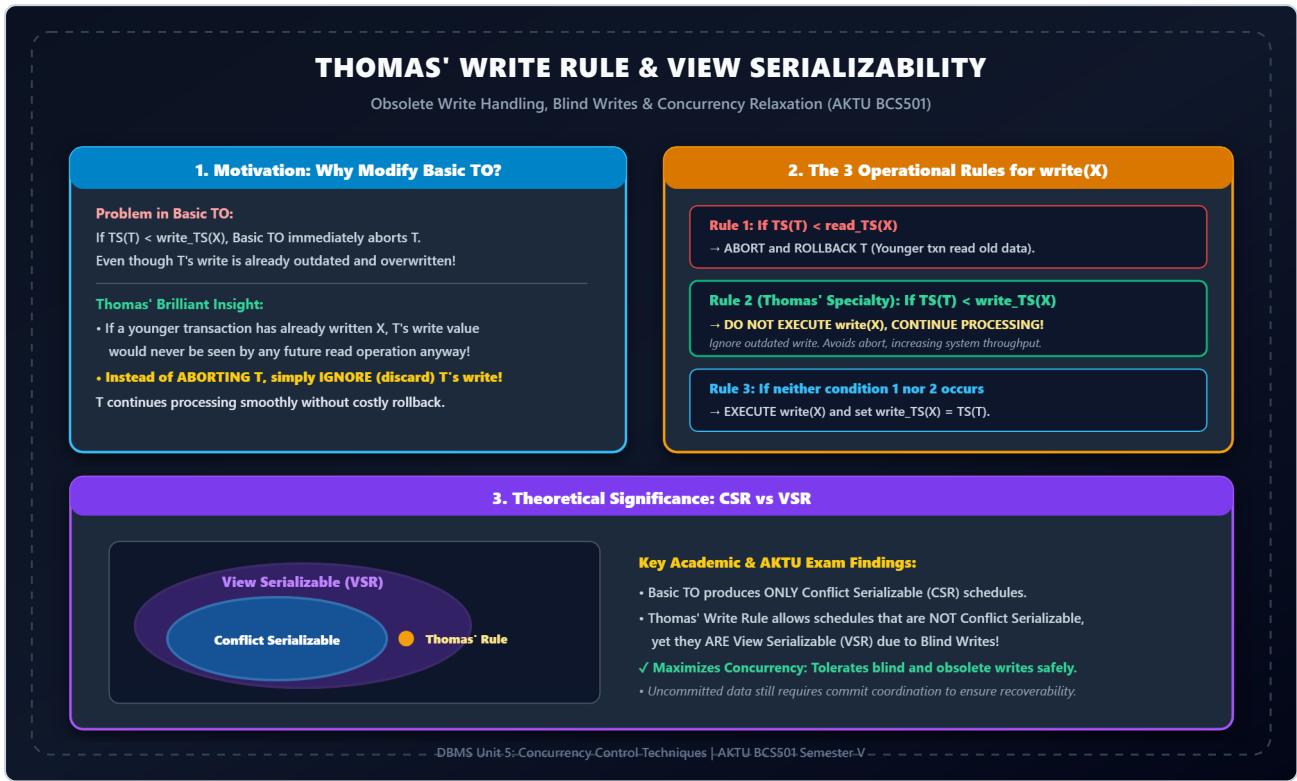


Figure 5.1: Thomas' Write Rule Decision Logic & View Serializability Scope

## 1. Why Thomas' Write Rule?

In Basic TO, if  $TS(T) < write\_TS(X)$ , T is immediately aborted. Thomas realized that if a younger transaction has already overwritten X, T's write is obsolete. Discarding the write rather than aborting T preserves View Serializability while boosting throughput.

## 2. The 3 Rules of Thomas' Write Rule

- **Rule 1:** If  $TS(T) < read\_TS(X) \rightarrow$  ABORT T.
- **Rule 2 (Thomas' Core):** If  $TS(T) < write\_TS(X) \rightarrow$  IGNORE write, CONTINUE processing!
- **Rule 3:** Otherwise, execute write and set  $write\_TS(X) = TS(T)$ .

### View Serializability Invariant

Thomas' Write Rule generates schedules that are strictly **View Serializable (VSR)**, even if they are not Conflict Serializable.



# Module 06: Multiversion Concurrency Control (MVCC) and MVTO

AKTU BCS501 | Unit 5: Concurrency Control Techniques

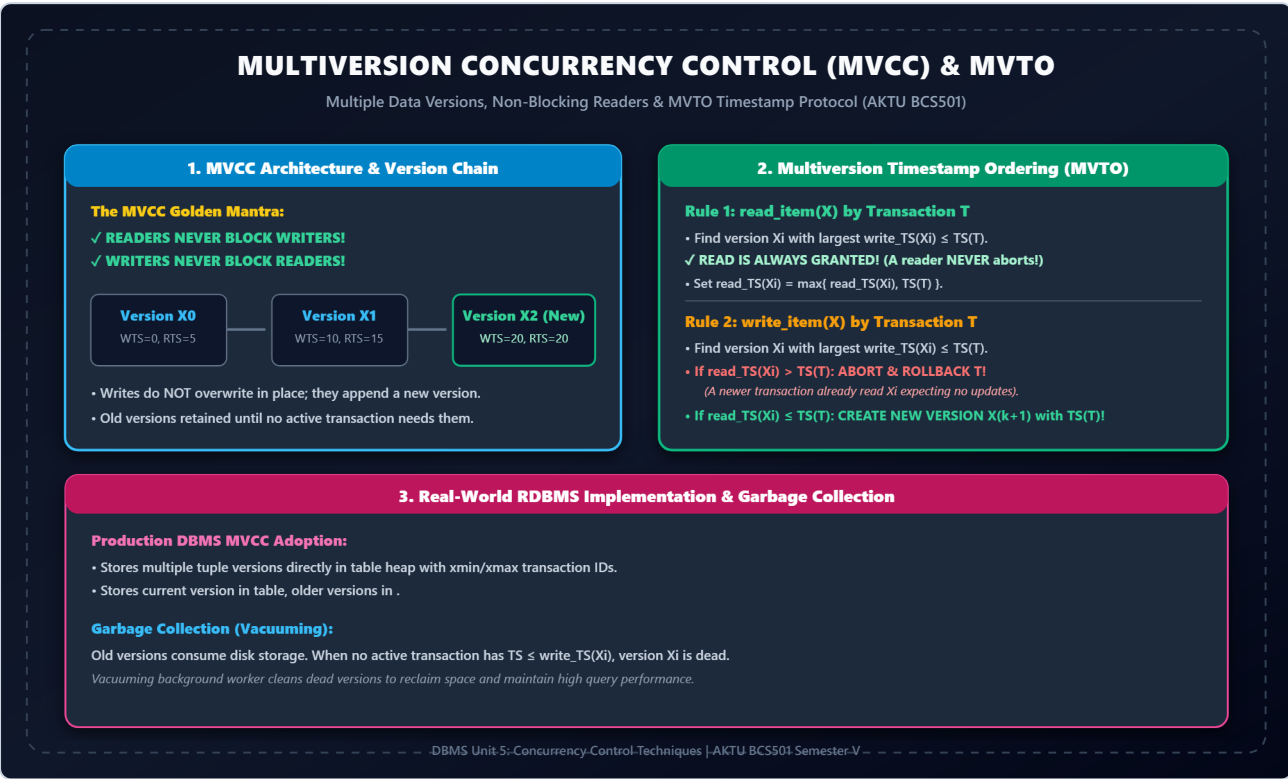


Figure 6.1: Multi-Version Timestamp Ordering Protocol & Version Chain

## 1. MVCC Architecture

In MVCC, writers append new versions rather than overwriting data in-place. Readers access older committed snapshots. Result: **Readers never block writers, and writers never block readers!**

## 2. MVTO Operational Logic

**read(X):** Reads the version Xi with maximal write\_TS(Xi) ≤ TS(T). Read is ALWAYS granted! Sets read\_TS(Xi) = max(read\_TS(Xi), TS(T)).

**write(X):** Finds Xi with maximal write\_TS(Xi) ≤ TS(T). If read\_TS(Xi) > TS(T) → ABORT T. Else creates new version with TS(T).

## AKTU BCS501 | Unit 5: Concurrency Control Techniques



MV2PL employs Read Locks (RL), Write Locks (WL), and Certify Locks (CL). Read and Write locks are fully compatible, allowing simultaneous concurrent reading and drafting.

Before committing, a writer transaction must upgrade its Write Locks to Certify Locks. Certify locks are mutually exclusive with all modes, ensuring strict commit-order serializability and eliminating cascading rollbacks.

# Module 08: Validation-Based Protocol (OCC) and Phantom Phenomena

AKTU BCS501 | Unit 5: Concurrency Control Techniques

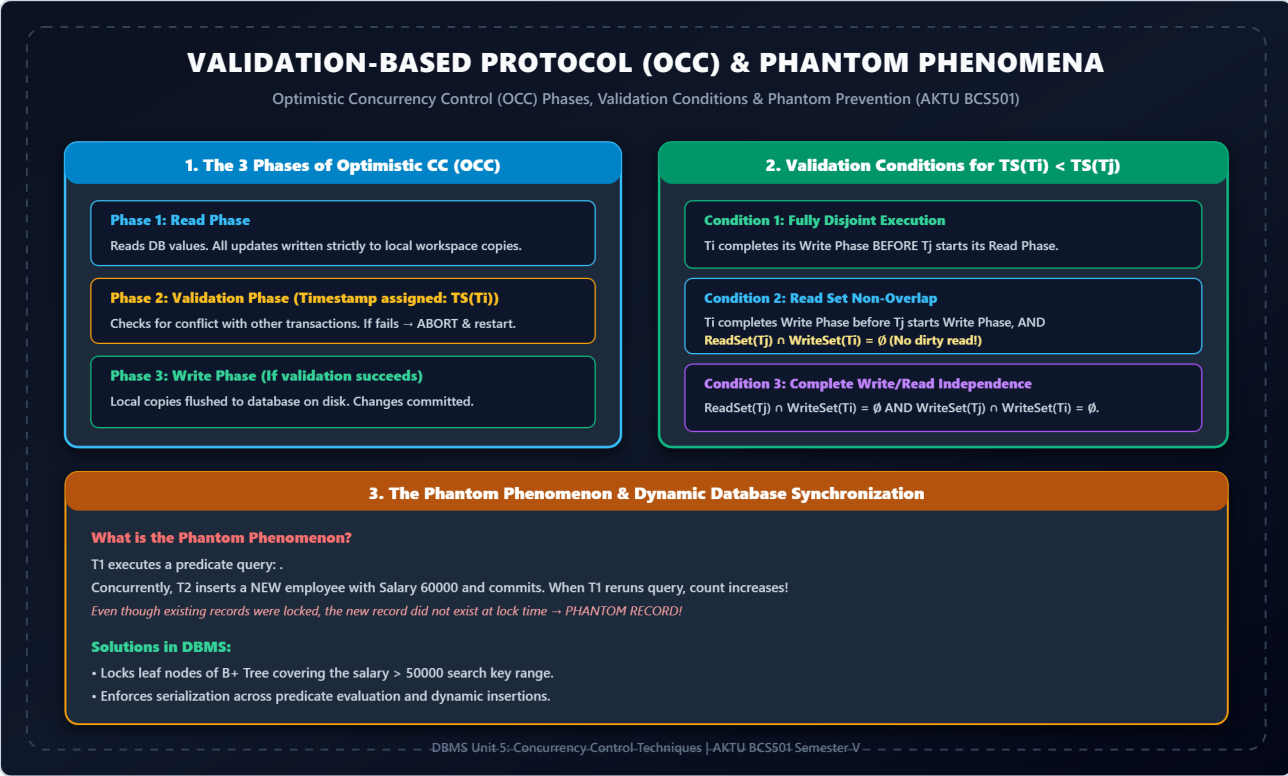


Figure 8.1: The 3 OCC Execution Phases, Validation Conditions & Phantom Prevention

## 1. The 3 Phases of OCC

- 1. **Read Phase:** Reads DB and buffers updates locally in memory.
- 2. **Validation Phase:** Checks conflict conditions; assigns TS(T).
- 3. **Write Phase:** Commits updates to disk if validation passes.

## 2. The Phantom Phenomenon

Predicate queries (e.g., SELECT COUNT(\*) WHERE Salary > 50k) can see phantom records if concurrent transactions insert new matching rows. Solved via **Index-Range Locking** on B+ Tree leaves or Predicate Locking.

# Module 09: Granularity of Data Items and Hierarchy

AKTU BCS501 | Unit 5: Concurrency Control Techniques

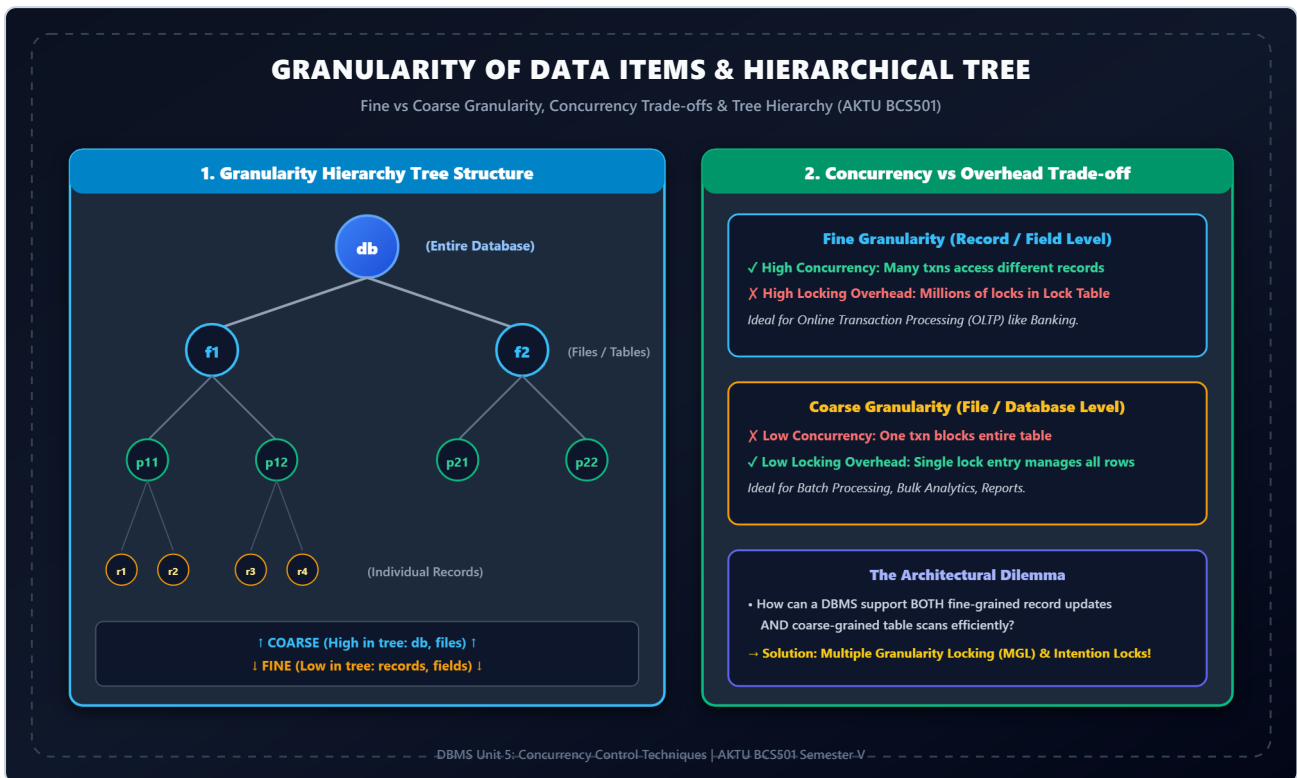


Figure 9.1: Granularity Tree Hierarchy & Concurrency vs Overhead Trade-offs

## 1. Granularity Definition

Granularity refers to the size of data items chosen for locking (database, files, pages, records, fields).

## 2. Concurrency vs Overhead Trade-off

**Fine Granularity (Record):** Maximizes concurrent access, but incurs heavy lock table overhead.

**Coarse Granularity (File/Database):** Minimal locking overhead, but restricts concurrency.

# Module 10: Multiple Granularity Locking and Intention Locks

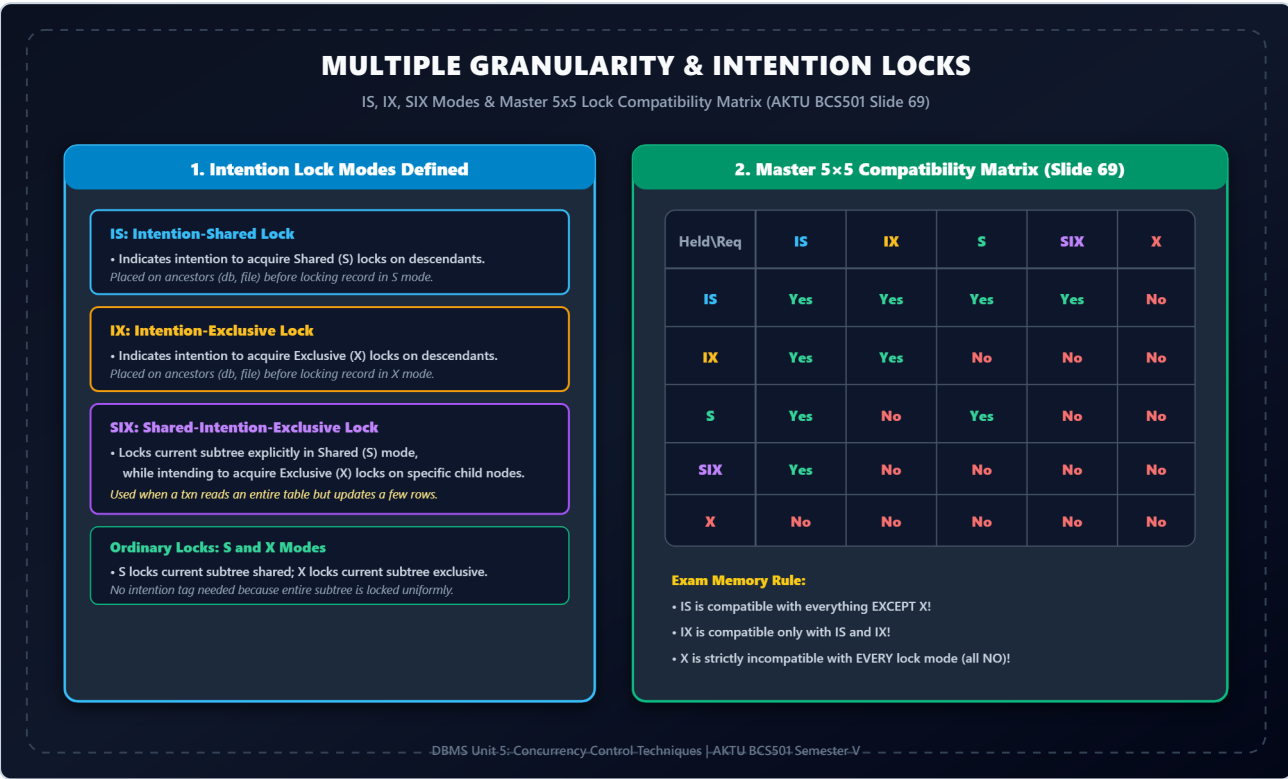


Figure 10.1: Intention Locks (IS, IX, SIX) & Master 5x5 Compatibility Matrix

## 1. Purpose of Intention Locks

Intention locks avoid full ancestor-descendant tree traversal when checking lock conflicts. Before locking a leaf in S or X mode, intention locks (IS or IX) are placed on all ancestor nodes.

## 2. The Master 5x5 Lock Compatibility Matrix

| Held \ Requested | IS  | IX  | S   | SIX | X  |
|------------------|-----|-----|-----|-----|----|
| IS               | Yes | Yes | Yes | Yes | No |
| IX               | Yes | Yes | No  | No  | No |
| S                | Yes | No  | Yes | No  | No |
| SIX              | Yes | No  | No  | No  | No |
| X                | No  | No  | No  | No  | No |

# Module 11: Multiple Granularity Locking Protocol Rules



Figure 11.1: The 6 Rules of Multiple Granularity Locking & Operational Tracing

## 1. The 6 Rules of MGL Protocol

1. Lock compatibility matrix must be adhered to.
2. Root of tree must be locked first.
3. Node locked in S/IS only if parent is locked in IS/IX.
4. Node locked in X/IX/SIX only if parent is locked in IX/SIX.
5. Lock only if no node has been unlocked (2PL compliance).
6. Unlock node only if none of its children are locked (Bottom-Up unlocking).

# Module 12: Recovery with Concurrent Transactions and 2PC

AKTU BCS501 | Unit 5: Concurrency Control Techniques

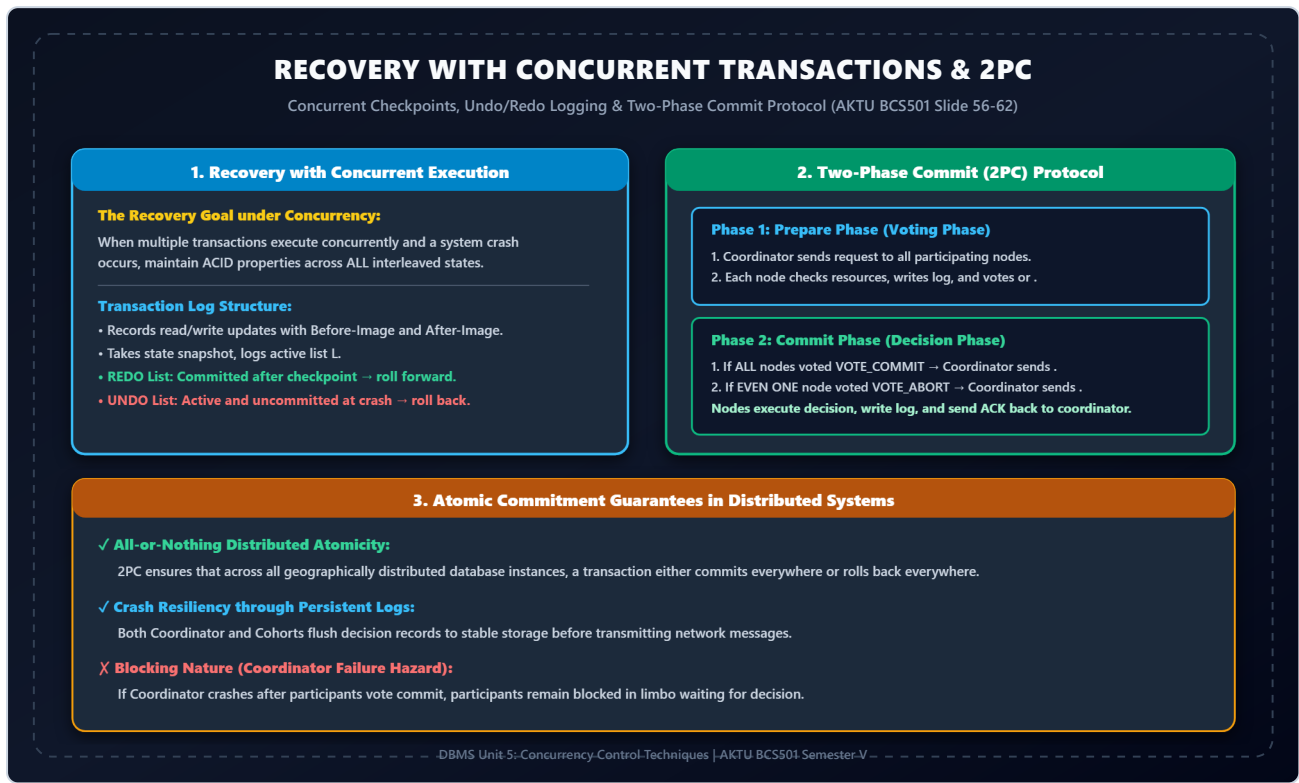


Figure 12.1: Concurrent Transaction Recovery, Checkpoints & 2PC Protocol

## 1. Concurrent Transaction Recovery

Maintains ACID properties during crashes. Active list L at checkpoint determines UNDO list (uncommitted active txns) and REDO list (committed txns after checkpoint).

## 2. Two-Phase Commit (2PC) Protocol

Phase 1 (Prepare): Coordinator sends PREPARE; nodes vote COMMIT or ABORT.

Phase 2 (Commit): If all vote yes, coordinator sends GLOBAL\_COMMIT; else GLOBAL\_ABORT.

# Module 13: Case Study of Oracle Concurrency Control

AKTU BCS501 | Unit 5: Concurrency Control Techniques

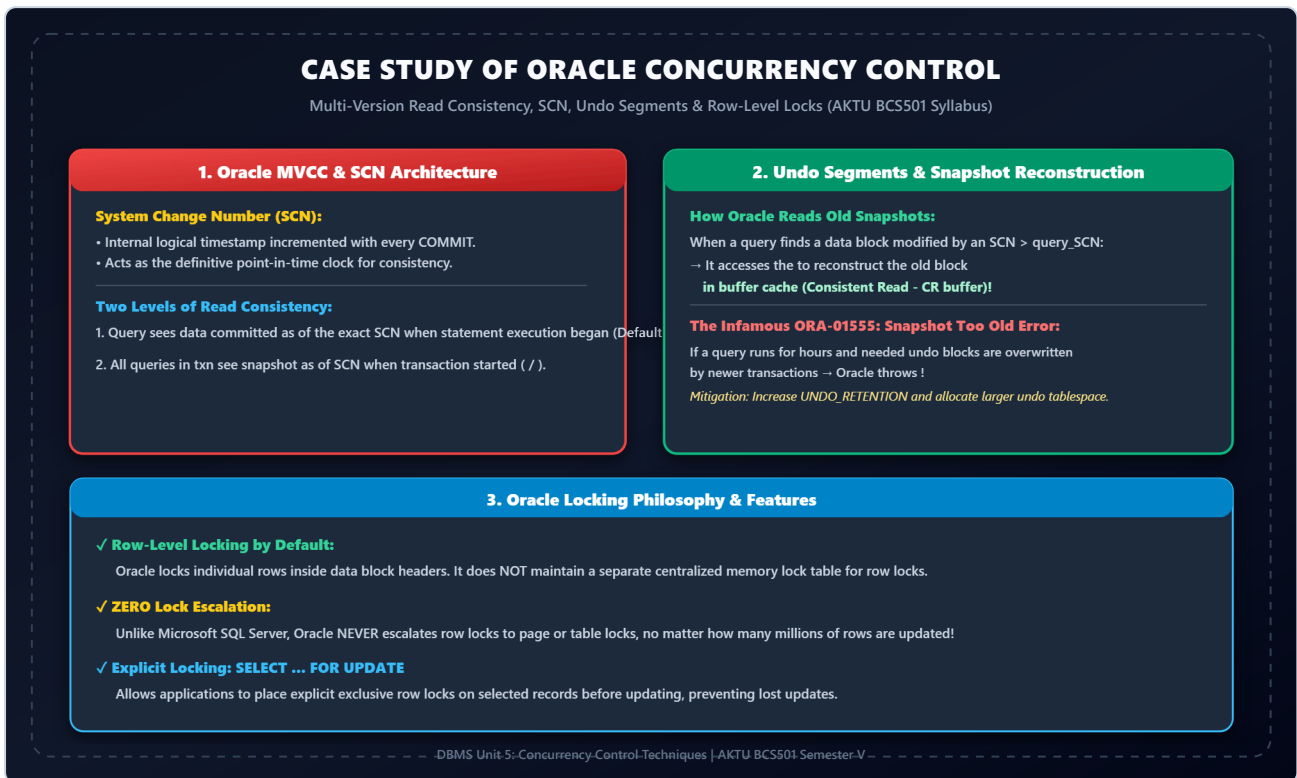


Figure 13.1: Oracle MVCC, System Change Number (SCN) & Undo Segment Reconstruction

## 1. Oracle Multi-Version Read Consistency

Oracle never allows readers to block writers or writers to block readers. Readers access historical snapshots reconstructed from Undo Segments based on the query's start SCN.

## 2. Key Oracle Features

- **SCN (System Change Number):** Logical clock incremented at every commit.
- **ORA-01555 (Snapshot Too Old):** Occurs when required undo blocks are overwritten.
- **No Lock Escalation:** Row locks are stored in block headers and never escalate to table locks.



## AKTU UNIT 5 MASTER EXAM DECISION FLOWCHART

Step-by-Step Problem Solving Algorithms for 10-Mark AKTU Concurrency Questions (BCS501)

**Step 1: Identify Concurrency Technique**  
Is question on Locking, Timestamps, MGL, or Oracle?

**Branch A: Locking & 2PL**

- Check Growing vs Shrinking Phase
- Any lock acquired after unlock? → Violates 2PL
- X-locks held till commit? → Strict 2PL
- All locks held till commit? → Rigorous 2PL
- ✓ Pre-locking all items? → Conservative 2PL

**Branch B: Timestamps & Thomas Rule**

- read(X):  $TS(T) < write\_TS(X)$  → ABORT
- write(X):  $TS(T) < read\_TS(X)$  → ABORT
- write(X) with  $TS(T) < write\_TS(X)$ :
  - Basic TO: ABORT T
  - Thomas Rule: IGNORE & CONTINUE (VSR)

**Branch C: Multiple Granularity (MGL)**

- Draw 5×5 Compatibility Matrix (IS, IX, S, SIX, X)
- Locking: Top-down from root (db)
- Unlocking: Bottom-up from leaves
- S requires IS/IX on parent
- X requires IX/SIX on parent

### AKTU BCS501 Past 5-Year Question Patterns

**Q1 (10 Marks): Explain Concurrent Control Mechanism & Compare 2PL vs Timestamp Ordering (2022-23)**  
→ Draw 2PL growing/shrinking curve + TO read/write rules. Mention Deadlocks in 2PL vs Deadlock-freedom in TO.

**Q2 (10 Marks): What is Multiple Granularity Locking? Explain Compatibility Matrix & Rules (2018-19, 2021-22)**  
→ Draw Granularity Tree (db → f → p → r), draw 5×5 matrix, and list the 6 rules of MGL protocol.

**Q3 (10 Marks): Explain Oracle Concurrency Control & Multi-Version Read Consistency (2020-21)**  
→ Detail SCN mechanism, Undo Segments / CR buffer reconstruction, ORA-01555 causes, and lack of lock escalation.